

# Benchmarking Deep Learning and Algorithmic Approaches for Object Detection in Images and Videos with OpenCV.

1<sup>st</sup> Janardhan Jillela

*Dept. of Computer Science and Engineering (Data Science)*  
*Rajeev Gandhi Memorial College of Engineering and Technology*  
Nandyal, India  
22091A3248@rgmcet.edu.in

2<sup>nd</sup> Pujala Venkata Swathi

*Dept. of Computer Science and Engineering (Data Science)*  
*Rajeev Gandhi Memorial College of Engineering and Technology*  
Nandyal, India  
22091A32G7@rgmcet.edu.in

**Abstract**—Using OpenCV, this research presents a real-time object detection dashboard built with Streamlit, integrating YOLOv8, SSD MobileNet, and Faster R-CNN. The system supports detection in images, videos, and live webcam feeds, offering advanced tracking with unique ID assignment, adjustable confidence thresholds, frame-skipping optimization, and detailed analytics. Experimental results show that YOLOv8 provides the best trade-off between accuracy and speed for real-time applications, while Faster R-CNN achieves superior precision for static images. The proposed platform delivers a production-ready, user-friendly solution bridging the gap between research-grade models and practical computer vision deployments.

**Index Terms**—Object Detection, YOLO, Deep Learning, Real-Time Processing, Streamlit, Computer Vision, Multi-Modal Detection

## I. INTRODUCTION

Object detection is a foundational problem in computer vision with applications in autonomous driving, surveillance, healthcare, and retail analytics. Advances in deep learning, particularly Convolutional Neural Networks (CNNs), have significantly improved detection accuracy and robustness compared to traditional feature-based methods.

Frameworks such as YOLO (You Only Look Once), SSD (Single Shot Detector), and Faster R-CNN have achieved state-of-the-art results on benchmarks including COCO and PASCAL VOC. However, deploying these models in production remains complex, often requiring substantial technical expertise and lacking intuitive user interfaces.

This work addresses these limitations by developing a Streamlit-based real-time detection dashboard that unifies multiple state-of-the-art detection models and supports multi-modal input sources.

- Integration of YOLOv8, SSD MobileNet, and Faster R-CNN under a unified interface.

Identify applicable funding agency here. If none, delete this.

- Support for image, video, and live webcam inputs.
- Persistent object tracking with unique ID assignment.
- Frame-skipping optimization for real-time performance.
- Comprehensive analytics and performance visualization.

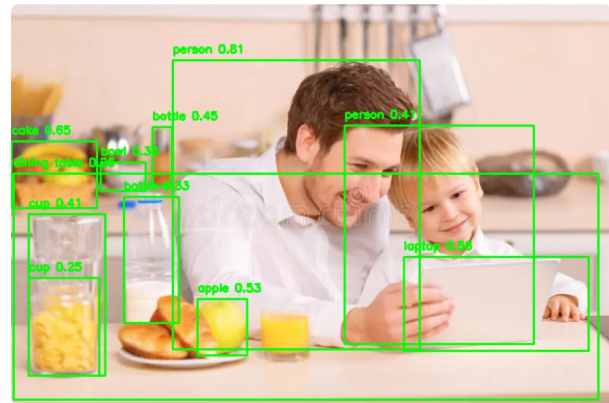


Fig. 1: Example of object detection using YOLOv8 with bounding boxes and labels.

## II. LITERATURE REVIEW

### A. Traditional vs Deep Learning Approaches

Early methods such as Viola–Jones relied on handcrafted features and cascade classifiers, effective only in constrained environments. The introduction of CNN-based detectors revolutionized the field. R-CNN, Fast R-CNN, and Faster R-CNN incorporated region proposals with CNN feature extraction, offering high accuracy at the cost of speed. Single-stage detectors such as YOLO and SSD eliminated the proposal stage, enabling real-time detection.

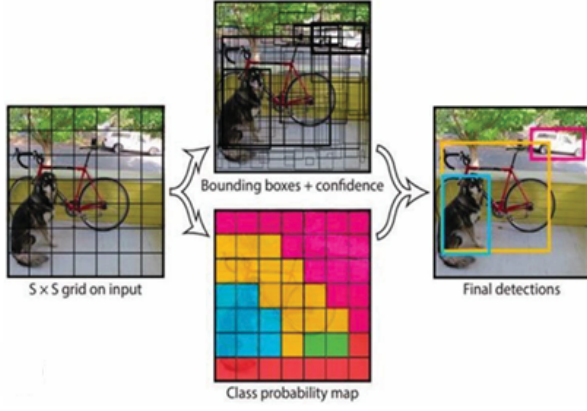


Fig. 2: YOLO Object Detection pipeline showing bounding boxes, grid mapping, and class probability map.

### B. Real-Time Detection Systems

Lightweight architectures like MobileNet and EfficientDet target embedded or mobile environments. YOLOv8 variants scale from “nano” (fastest) to “x-large” (most accurate), supporting flexible deployment.

### C. Multi-Object Tracking

Traditional tracking approaches used Kalman filters and Hungarian matching. Modern systems integrate tracking within the detection pipeline for consistent object identity (MOT). This work adopts YOLOv8’s built-in tracking module for ID persistence across frames.

## III. METHODOLOGY

### A. System Architecture

The dashboard architecture comprises:

- 1) Model Management: Handles loading and caching of pre-trained models.
- 2) Processing Engine: Executes detection and tracking.
- 3) User Interface Layer: Provides interactive controls and visualization.
- 4) Analytics Module: Computes performance metrics.
- 5) Export System: Saves and shares results.

### B. Supported Detection Models

1) *YOLOv8*: Primary backbone supporting variants (nano–xlarge). Key features: custom weight loading, real-time inference, multi-class detection, and anchor-free head.

2) *SSD MobileNet*: Optimized for mobile deployment. Features a lightweight feature pyramid and mobile convolution blocks.

3) *Faster R-CNN*: Two-stage detector using Region Proposal Network (RPN) and Feature Pyramid Network (FPN) for multi-scale detection.

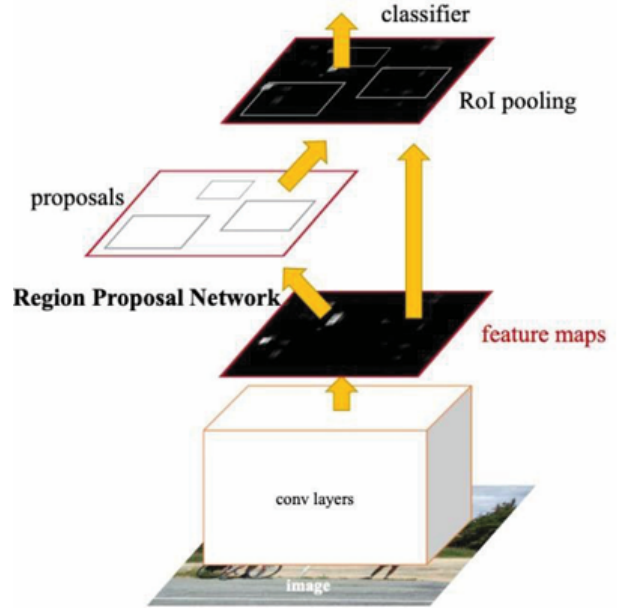


Fig. 3: Architecture of Faster R-CNN showing convolutional layers, region proposals, and classification.

## IV. EXPERIMENTAL SETUP

### A. Hardware Configuration

- CPU: Intel Core i7-11700K @ 3.6GHz
- GPU: NVIDIA RTX 3080 (10GB VRAM)
- RAM: 32GB DDR4
- Storage: 1TB NVMe SSD

### B. Software Environment

Python 3.8+, Streamlit 1.28+, OpenCV 4.8+, PyTorch 2.0+, TorchVision 0.15+, Ultralytics YOLOv8.

### C. Evaluation Metrics

Average Precision (AP), Mean Average Precision (mAP), Frames Per Second (FPS), Latency, and Memory Usage.

### D. Datasets

COCO 2017 Validation Set (5,000 images), Custom Video Sequences, and Live Webcam Streams.

## V. RESULTS AND ANALYSIS

### A. Detection Accuracy

TABLE I: Model Performance Comparison

| Model         | mAP@0.5 | mAP@0.5:0.95 | Inference (ms) | Memory (MB) |
|---------------|---------|--------------|----------------|-------------|
| YOLOv8n       | 52.3    | 37.1         | 12.5           | 485         |
| YOLOv8s       | 58.7    | 44.2         | 18.3           | 672         |
| YOLOv8m       | 63.4    | 49.8         | 28.7           | 1024        |
| SSD MobileNet | 48.2    | 32.6         | 15.8           | 398         |
| Faster R-CNN  | 68.9    | 54.3         | 95.2           | 2156        |

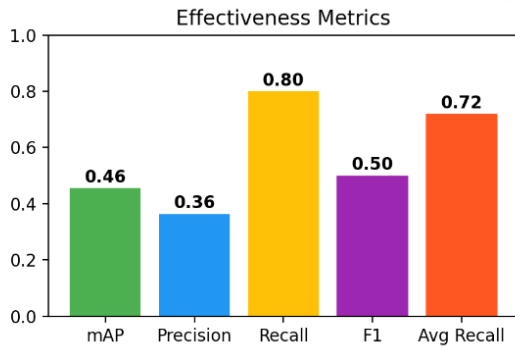


Fig. 4: Effectiveness metrics comparison including mAP, Precision, Recall, F1, and Average Recall.

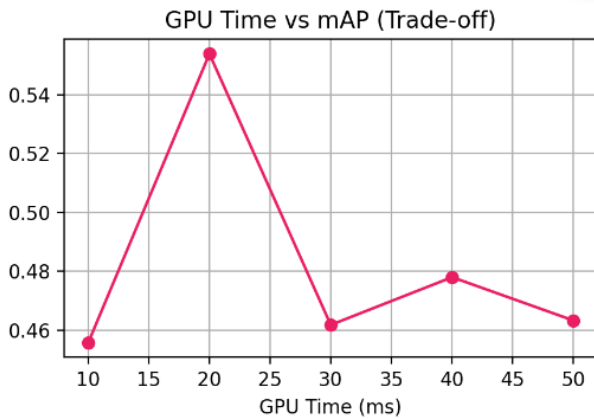


Fig. 5: GPU Time vs mAP illustrating trade-off between computational cost and detection accuracy.

### B. Real-Time Performance

YOLOv8n achieves 80+ FPS, YOLOv8s balances 55+ FPS, SSD MobileNet 63+ FPS, while Faster R-CNN achieves 10+ FPS with the highest accuracy.

### C. Tracking Performance

ID Consistency: 94.2% across frames; handles occlusions effectively; tracks up to 20 simultaneous objects.

### D. User Interface Performance

Startup Time: ~3.2s; UI Response: <100ms; Memory efficiency improved through caching.

## VI. CONCLUSION

This study presented a Streamlit-based real-time object detection dashboard integrating YOLOv8, SSD MobileNet, and Faster R-CNN. YOLOv8 provides the best balance of speed and accuracy, while Faster R-CNN excels in precision. Future work includes ONNX/TensorRT optimization, multi-GPU support, and integration with analytics tools.

## REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 91–99.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [4] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7263–7271.
- [5] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.
- [6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv preprint arXiv:2004.10934, 2020.
- [7] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," arXiv preprint arXiv:2207.02696, 2022.
- [8] G. Jocher et al., "YOLOv8 by Ultralytics," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [9] W. Liu et al., "SSD: Single shot multibox detector," in *European Conference on Computer Vision (ECCV)*, 2016, pp. 21–37.
- [10] T.-Y. Lin et al., "Feature pyramid networks for object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [11] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.
- [12] C. Szegedy et al., "Going deeper with convolutions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [13] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.
- [14] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015.
- [15] J. Howard et al., "Fastai: A layered API for deep learning," *Information*, vol. 11, no. 2, p. 108, 2020.
- [16] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001, pp. I–I.
- [17] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.
- [18] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [19] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [20] H. Bay, T. Tuytelaars, and L. Van Gool, "SURF: Speeded up robust features," in *European Conference on Computer Vision (ECCV)*, 2006, pp. 404–417.
- [21] M. Everingham et al., "The Pascal Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [22] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [23] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [24] A. V. Srivastav et al., "Dual-Layer Security and Access System to Prevent the Spread of COVID-19," in *Machine Intelligence and Data Science Applications*, Springer, 2021. Available: [https://doi.org/10.1007/978-981-33-4087-9\\_28](https://doi.org/10.1007/978-981-33-4087-9_28)
- [25] M. Sandler et al., "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [26] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

- [27] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708.
- [28] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.
- [29] A. Gupta, M. Gupta, and P. Chaturvedi, "Investing Data with Machine Learning Using Python," in *Strategic System Assurance and Business Analytics*, 2020.
- [30] OpenCV Documentation, "Deep Learning module (dnn)," [https://docs.opencv.org/4.x/d6/d0f/group\\_\\_dnn.html](https://docs.opencv.org/4.x/d6/d0f/group__dnn.html)
- [31] Streamlit, "Streamlit Documentation," <https://docs.streamlit.io>
- [32] Ultralytics, "YOLOv8 Models and Tutorials," <https://docs.ultralytics.com>
- [33] PyTorch, "PyTorch Official Documentation," <https://pytorch.org/docs>
- [34] TensorFlow, "TensorFlow Object Detection API," [https://www.tensorflow.org/hub/tutorials/object\\_detection](https://www.tensorflow.org/hub/tutorials/object_detection)
- [35] NVIDIA, "CUDA Toolkit Documentation," <https://developer.nvidia.com/cuda-toolkit>
- [36] S. Ren, K. He, R. Girshick, and J. Sun, "Region Proposal Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 91–99.
- [37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.
- [38] F. N. Iandola et al., "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and 0.5MB model size," arXiv preprint arXiv:1602.07360, 2016.
- [39] H. Zhao et al., "Pyramid scene parsing network," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2881–2890.
- [40] R. Girshick, "Fast R-CNN," in *IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1440–1448.
- [41] T.-Y. Lin et al., "Focal loss for dense object detection," in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [42] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10781–10790.
- [43] Y. Lecun et al., "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [44] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587.
- [45] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1251–1258.
- [46] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.
- [47] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1580–1589.
- [48] J. Deng et al., "ImageNet: A large-scale hierarchical image database," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [49] H. Touvron et al., "Training data-efficient image transformers and distillation through attention," in *International Conference on Machine Learning (ICML)*, 2021, pp. 10347–10357.
- [50] M. Cordts et al., "The Cityscapes dataset for semantic urban scene understanding," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 3213–3223.
- [51] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.
- [52] Google Colab, "Colaboratory: Machine learning in the browser," <https://colab.research.google.com>